Mathematical Notes on Neural Networks

From computing the outputs to gradient descent

These notes explore the mathematics of how a feed-forward neural network works and how it is trained: how neurons are combined, how the loss function is chosen, and how derivatives are computed and used to update the parameters. They are built around the Inside the Network application: the aim is to understand the computations and the reasons behind them, for the kind of networks considered here, rather than to offer a general treatment of machine learning.

The main text follows the sequence of prediction, loss, and learning. The appendices cover additional topics, namely temperature, the information-theoretic meaning of cross-entropy, and some numerical and implementation details; they can be read at the points where the text refers to them.

Contents

1. From inputs to the function computed by the network 2
1.1 A function that depends on parameters 4
1.2 What a neuron computes 5
1.3 Layers and notation 5
1.4 The forward pass 6
2. The role of activation functions 7
2.1 Why nonlinearity is needed 7
2.2 Sigmoid, hyperbolic tangent, and ReLU 7
3. From the network's output to the loss function 8
3.1 Prediction and decision 8
3.2 The softmax 9
3.3 Subtracting the maximum 9
3.4 Cross-entropy 9
3.5 The connection with maximum likelihood 11
4. Gradient descent 11
5. Backpropagation 12
5.1 The δ term 12
5.2 Derivative with respect to a weight and a bias 13
5.3 The δ term in the output layer 13
5.4 The δ term in a hidden layer 15
5.5 The procedure for a single example 16
6. How the loss function affects the gradient 16
6.1 Comparison with the squared error 16
6.2 An example with three classes 17
7. From a single example to the dataset 18
7.1 The average gradient and sampling 18
7.2 Epochs and mini-batches 18
8. From training to generalization 19
8.1 The scale of the inputs 19
8.2 Why the weights are initialized at random 19
8.3 Training loss and loss on new data 20
9. From index notation to matrix form 20
9.1 Forward pass 20
9.2 Backward pass in the hidden layers 21
9.3 Derivatives with respect to the parameters 21
9.4 Extension to mini-batches 21

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Appendix A. Temperature and the Boltzmann distribution 23
Appendix B. Entropy and information 24
B.1 Information and surprise 24
B.2 Shannon entropy and Kullback–Leibler divergence 24
Appendix C. Computing cross-entropy numerically 25
Appendix D. One epoch of random reshuffling to first order 26
Appendix E. Correspondence with the application's code 27
Further reading 28

1. From inputs to the function computed by the network

A neural network is a mathematical model for making predictions. It learns from examples that have already been solved, approximates the rule that links the inputs to the result, and then applies that rule to new cases.

As an example, take one of the problems in the simulator, Catching the bus: the bus leaves in T minutes and we are D meters from the stop; will we catch it? Each training example consists of a pair of inputs, the values of T and D, and the corresponding result: 1 if the bus was caught, 0 if it was missed. Here the rule is known (walking at 80 m/min, we catch the bus when $D < 80T$), so we can compare it with the rule the network learns. In real problems, by contrast, the rule is unknown or too complicated to write down explicitly, and that is precisely why we rely on examples. Recognizing a handwritten digit in an image is a problem of the same kind: the inputs are the pixel intensities and the result is the digit shown.

A familiar starting point is fitting a function to experimental data: we choose a family of functions, for example the lines $y = mx + q$, and determine its parameters, here m and q, so that the function approximates the given points. A neural network does the same thing with a much richer family of functions, whose parameters are called weights and biases. Learning means gradually adjusting these parameters on the basis of the training examples. This is called supervised learning because each example also includes the expected result, which from now on we will call the target.

The functions in the family are built by connecting simple computational units called neurons. The name is inspired by biology, but here it simply denotes a mathematical unit: each neuron combines the values it receives, according to its own weights and bias, and produces a new value (§1.2). Neurons are arranged in layers: the first receives the inputs, the last produces the output values, or outputs, from which the prediction is obtained; the layers in between are called hidden layers. The values produced by one layer become the inputs to the next, so the function computed by the network is a composition of simpler functions.

Training is a cycle that is repeated many times (Figure 1). The parameters are given initial values: in the application, the weights are chosen at random and the biases are set to zero (§8.2). For each example, the network receives the inputs, computes the outputs, and derives a prediction from them; the prediction is compared with the target by means of a loss function, whose value measures how far the prediction is from the expected result (§3). The derivatives of the loss with respect to the weights and biases indicate how to change each parameter in order to reduce the loss (§4 and §5). The updated parameters become the current parameters for the next step, and the cycle is repeated many times over all the training examples (§7). The rule linking the inputs to the result is therefore never given to the network: it is approximated through these updates.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

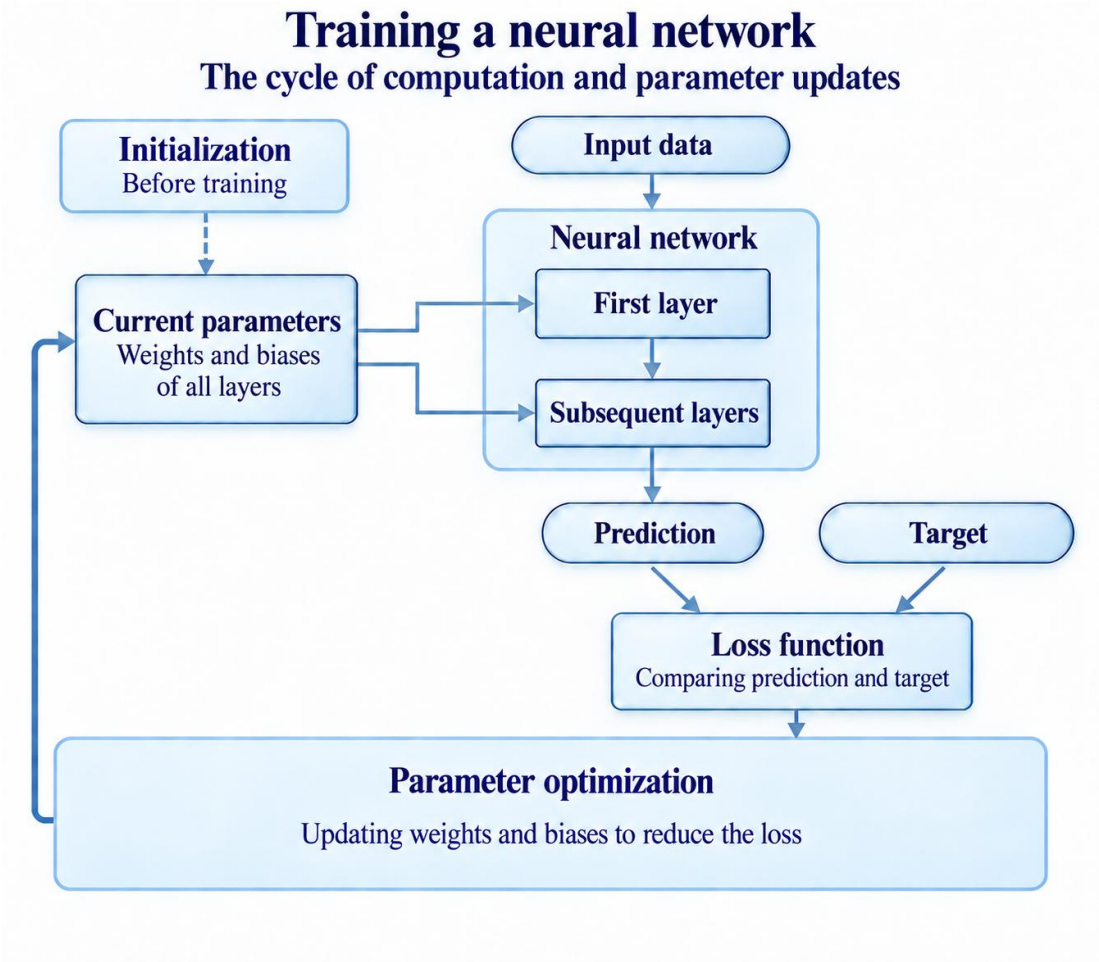


Figure 1. The training cycle. The current parameters determine the function computed by the network; the loss function compares the prediction with the target and drives the update of the parameters, which are then used in the next step.

Once training is over, the weights and biases stay fixed and the network is used like any other function: it receives new inputs and computes the outputs with the learned parameters (Figure 2). In the bus problem, for example, the trained network also gives an answer for pairs of values of T and D that did not appear among the examples. A good fit to the training examples does not, however, guarantee correct predictions on new data: this ability, called generalization, must be checked separately, on examples not used during training (§8.3).

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

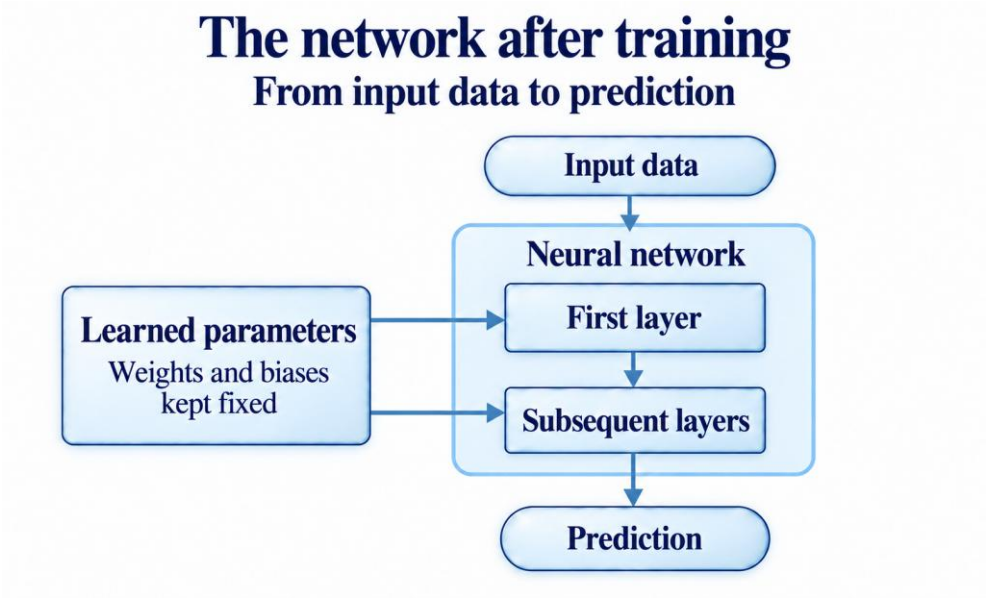


Figure 2. The network after training. With the learned parameters kept fixed, the network turns the input data into a prediction: there is no longer any comparison with a target, and no parameter update.

In the simulator the network is deliberately small, so that the weights, biases, values computed by the neurons, and loss can be observed in detail. The networks used in real applications follow the same principles, with a vastly larger number of parameters.

1.1 A function that depends on parameters

Once the structure of the network is fixed (how many layers, how many neurons in each layer, and which activation functions: §1.2 and §2), the function computed by the network depends only on the values of the parameters. We denote it by $F_\theta(x)$: x is the vector of inputs, and θ is the vector collecting all the weights and biases. If the output is a single number, we denote the prediction by $\hat{y} = F_\theta(x)$; in problems with several classes, the output is instead a vector, with one component for each class (§3). As θ varies, we obtain a family of functions, just as varying m and q gives the family of lines $y = mx + q$.

The distinction between x and θ lets us describe precisely the two phases just introduced, in which the two variables swap roles. When the trained network is used, θ is fixed and x varies: F_θ is a function of the inputs (Figure 2). During training, by contrast, the examples are fixed and θ varies: the loss on the training examples becomes a function of the parameters alone. If there are N examples and $E_r(\theta)$ is the loss on example r, we consider the average loss

$$\bar{E}(\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r\,(\theta).$$

Training the network means looking for values of θ for which $\bar{E}(\theta)$ is small, that is, choosing from the family a function that agrees well with the examples. The same weights and biases are used for all the examples: we are looking for a single function, not a different function for each example.

For the line that best fits a set of points in the least-squares sense, the optimal coefficients are given by an explicit formula. For a neural network this is generally not possible: we start from initial values of the parameters and improve them step by step, using the derivatives of $\bar{E}$ with respect to the parameters. This is the method of gradient descent (§4); backpropagation (§5) is the procedure that computes these derivatives efficiently.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

In the networks considered in these notes, called feed-forward networks, connections always go from one layer to the next, from the inputs toward the output: F_θ is therefore the composition of the transformations computed by the successive layers, as shown in Figure 3 in §1.4.

1.2 What a neuron computes

A neuron combines the values it receives through a weighted sum, adds a constant term called the bias, and applies an activation function. With two inputs:

$$z = w_1 x_1 + w_2 x_2 + b, \qquad a = f(z).$$

The value z is called the pre-activation; a is the activation passed on to the neurons in the next layer. The weights determine how much each input contributes to z; the bias shifts its value even when the inputs are fixed. The name "neuron" here refers only to this simple computational unit, with no claim to biological meaning.

The first operation is affine: a linear transformation plus a constant term. With two inputs, if at least one weight is nonzero, the equation $w_1 x_1 + w_2 x_2 + b = 0$ describes a line. If the activation is the sigmoid (§2.2) and the classification threshold is $1/2$, this line separates the two decision regions, because $\sigma(0) = \frac{1}{2}$ and the sigmoid is increasing, so $z < 0$ implies $\sigma(z) < \frac{1}{2}$ and $z > 0$ implies $\sigma(z) > \frac{1}{2}$.

1.3 Layers and notation

We number the layers from 0 to L. Layer 0 contains the inputs; layer L is the output layer. We denote by n_ℓ the number of nodes in layer ℓ; the nodes of layer ℓ are numbered from 1 to n_ℓ.

We denote by $x = (x_1, \dots, x_{n_0})^T$ the column vector collecting the inputs of a single example, and set $a_i^{(0)} = x_i$. We will also use the following convention:

Symbol	Meaning
$a_i^{(\ell)}$	value of node i in layer ℓ; for $\ell = 0$, $a_i^{(0)} = x_i$
$w_{ij}^{(\ell)}$	weight of the connection from node j in layer $\ell - 1$ to neuron i in layer ℓ
$b_i^{(\ell)}$	bias of neuron i in layer ℓ
$z_i^{(\ell)}$	pre-activation value of neuron i in layer ℓ
f_ℓ	activation function used in layer ℓ

For $a_i^{(\ell)}$ the value $\ell = 0$ is also allowed; for weights, biases, pre-activations, and activation functions, it is understood that $\ell = 1, \dots, L$. The transitions between layers therefore go from $\ell - 1$ to ℓ, with $\ell = 1, \dots, L$.

In problems with a single output, y denotes the target value; in the multiclass case, c denotes the index of the correct class and t_i the components of the *one-hot* target (1 for the correct class, 0 for all the others).

The layers between the input and the output are called hidden layers. The data assign no target values to their activations: the parameters are changed on the basis of the final loss of the network. In the networks considered here, each neuron receives all the activations of the previous layer; such layers are called fully connected.

In the notation $w_{ij}^{(\ell)}$, the index i identifies the destination neuron, j the source neuron, and ℓ the destination layer. For example, $w_{21}^{(2)}$ connects the first neuron of the first hidden layer to the second neuron of the next layer. The superscript in parentheses denotes the layer, not a power. The correspondence with the arrays in the code is given in Appendix E.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

In the application's "Getting started: what is a neural network?" panel, the layer index and the destination neuron index are left implicit when describing a single neuron, and the incoming connections are numbered with the index i: the computation is therefore written as $z = \sum_i w_i\, a_i + b$. The index i in the panel thus corresponds to the index j used in these notes.

1.4 The forward pass

For $\ell = 1, \dots, L$, neuron i in layer ℓ receives the values $a_j^{(\ell-1)}$ from all the nodes of the previous layer and first computes the weighted sum

$$z_i^{(\ell)} = \sum_{j=1}^{n_{\ell-1}} w_{ij}^{(\ell)}\, a_j^{(\ell-1)} + b_i^{(\ell)} \tag{1}$$

then, in the layers where the activation is applied component by component, it applies the activation function:

$$a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big). \tag{2}$$

In the application's multiclass case, the output layer instead applies the softmax to the whole vector of pre-activation values. This function will be introduced in §3.2. The forward pass, that is, the computation of the network's output from the inputs, repeats these two operations layer by layer up to the output. For a problem with a single output, the output is

$$\hat{y} = a_1^{(L)}.$$

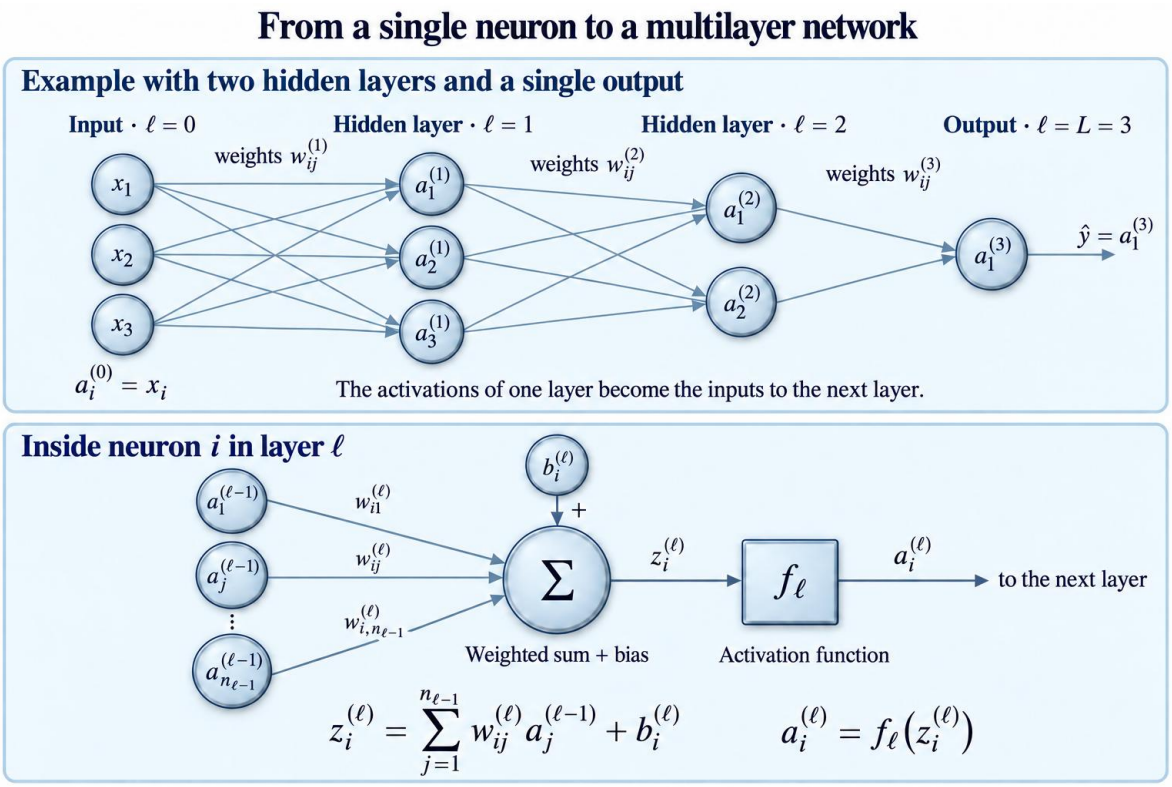


Figure 3. A network with three inputs, two hidden layers, and a single output. The lower panel shows the computation performed by a generic neuron. The biases, omitted from the upper diagram for readability, are present in every hidden and output neuron. The softmax transformation of §3.2, by contrast, acts on the whole output vector.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

2. The role of activation functions

2.1 Why nonlinearity is needed

Adding more layers is not enough, by itself, to obtain more general functions. If all the hidden-layer activations were the identity $f(z) = z$, a composition of affine transformations would remain affine. This can already be seen with two layers, writing the weighted sums in matrix form:

$$v = Ax + b, \qquad u = Cv + d = CAx + (Cb + d).$$

Here A and C are weight matrices, b and d are bias vectors, and x, v, and u are vectors of values. The result still has the form "matrix times input plus constant vector". Repeating the argument, all the affine layers can be reduced to a single affine transformation.

A final sigmoid or softmax does not remove this limitation of the hidden layers: in binary classification the boundary is still a hyperplane (a line in the case of two inputs); in the multiclass case, the equalities between pairs of output pre-activations are still affine equations. Nonlinear activations in the hidden layers, on the other hand, make it possible to build transformations for which this reduction is impossible.

2.2 Sigmoid, hyperbolic tangent, and ReLU

The application lets you compare three activation functions in the hidden layers; their graphs are shown in Figure 4. Their derivatives will appear directly in the backpropagation formulas (§5).

Function	Definition	Derivative
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$\sigma'(z) = \sigma(z)(1-\sigma(z))$
Hyperbolic tangent	$\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$\tanh'(z) = 1-\tanh^2(z)$
ReLU	$f(z) = \max(0, z)$	0 for $z < 0$; 1 for $z > 0$

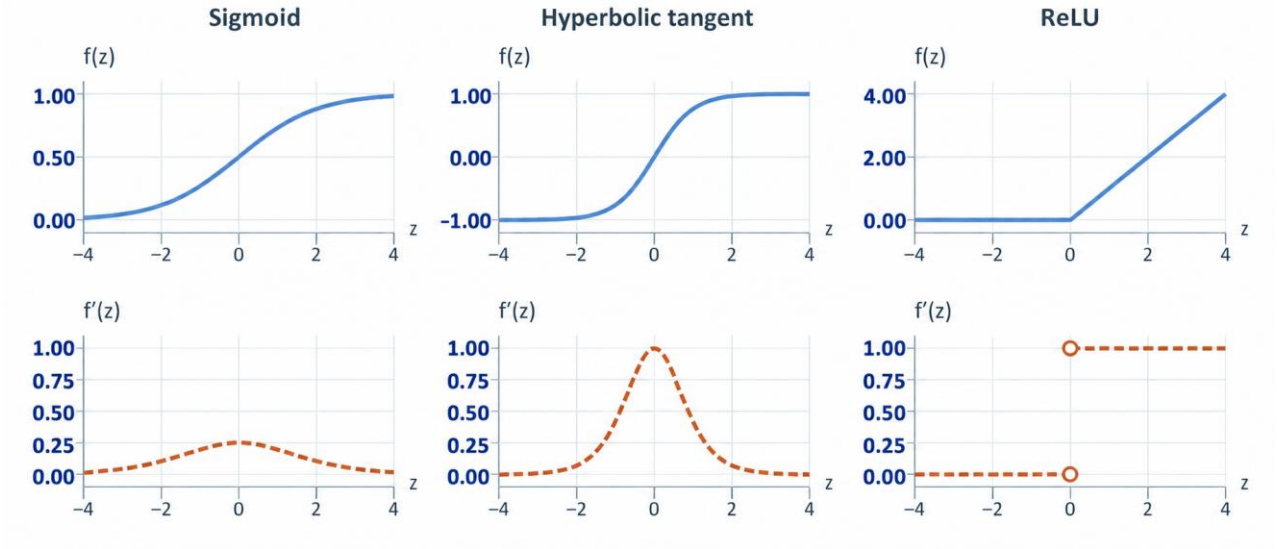


Figure 4. Activation functions (top) and their derivatives (bottom). The range of z is the same in all the graphs, and the vertical scale is the same for the three derivatives. For the ReLU, the open circles indicate that the derivative is not defined at $z = 0$.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

The sigmoid takes values in $(0,1)$, the hyperbolic tangent in $(-1,1)$. The two functions are related by the identity $\tanh(z) = 2\sigma(2z) - 1$: the graph of the hyperbolic tangent is obtained from that of the sigmoid by rescaling the axes and shifting it vertically. Both flatten out when $|z|$ is large: their derivatives tend to zero, and we say that they saturate. Unlike the sigmoid, the hyperbolic tangent also allows negative activations.

The ReLU (rectified linear unit) returns zero for negative arguments and leaves positive ones unchanged. It is piecewise affine, but not affine over its whole domain: it is precisely this change of expression that makes it possible to build richer transformations. It is not differentiable at $z = 0$; by convention, the application's code uses the value zero in place of the derivative there. The differentiation formulas should therefore be read as valid where the derivatives exist, with this convention at the points of non-differentiability.

The choice of activation affects both the functions that can be represented and the computation of the gradients. Saturation can weaken the derivatives propagated through several layers; a ReLU, on the other hand, has derivative one for $z > 0$, but passes no gradient through the branch $z < 0$. If the pre-activation is negative for all the training examples, the gradients with respect to the incoming weights and the bias are zero: as long as this condition persists, gradient descent does not change these parameters. In layers after the first, however, updates to the earlier layers can change the neuron's inputs and reactivate it. There is therefore no choice that automatically makes training easy in every situation.

3. From the network's output to the loss function

3.1 Prediction and decision

The transformation in the output layer depends on the type of problem. The table summarizes the choices made in the application; the output transformation can differ from the one used in the hidden layers.

Problem	Network output	Interpretation and decision
Regression	A value $\hat{y} = z_1^{(L)}$	Numerical prediction, no classification threshold
Binary classification	A value $a = \sigma(z_1^{(L)})$	Estimated probability of class 1; class 1 if $a \geq 1/2$, class 0 otherwise
Multiclass classification	A vector p produced by the softmax	Estimated class probabilities; a component with the largest value is chosen

In problems with a single output, we will also write $a = \hat{y} = a_1^{(L)}$ and $z = z_1^{(L)}$ for brevity: a and $\hat{y}$ thus denote the same value, while y remains the target.

The threshold and the choice of the maximum turn the outputs into a decision. The loss function, on the other hand, compares the numerical output with the target and guides training. For regression and binary classification, the application uses the squared error of a single example:

$$E = \frac{1}{2}(\hat{y} - y)^2.$$

The factor $1/2$ simplifies the derivative without changing the minimum points. In regression the loss has no upper bound, whereas in the binary case, with $\hat{y} \in [0,1]$ and $y \in \{0,1\}$, it is at most $1/2$. For multiclass classification, cross-entropy will be used instead; the comparison between the two choices, including the different convention for the vector squared error, is taken up again in §6.1.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

3.2 The softmax

In the multiclass case, the output layer contains $K = n_L$ neurons, one per class. For brevity, we write $z_i = z_i^{(L)}$ for the output pre-activations, often called logits. The softmax transforms them into the components

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \dots, K.$$

Each exponential is positive; dividing each of them by their sum gives $p_i > 0$ and $\sum_{i=1}^{K} p_i = 1$. Each output depends on all the pre-activations, because the denominator is shared. The softmax is therefore a transformation of the whole vector, as anticipated in §1.4.

The order of the values is preserved: if $z_i > z_j$, then $p_i > p_j$. More precisely,

$$\frac{p_i}{p_j} = e^{z_i - z_j}.$$

The relative probabilities therefore depend on the differences between the pre-activations: adding the same constant to all the components does not change the distribution. The softmax returns all the probabilities, and the application then picks the predicted class as the one with the highest probability.

For example, with $z = (2{,}0, -1)$ we get $p \approx (0.843795, 0.114195, 0.042010)$. These values are used again in §6.2 to compute the loss and the gradient.

The softmax components therefore have the mathematical properties of a probability distribution, but they do not necessarily reflect how often predictions are actually correct: among all the predictions made with confidence 0.9, for instance, the fraction that are correct may be quite different from 90%.

3.3 Subtracting the maximum

The previous formula can cause problems on a computer: for large positive values, the exponential can exceed the largest representable number (overflow). To avoid this, we set $z_{\max} = \max(z_1, \dots, z_K)$ and use the equivalent form

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_{j=1}^{K} e^{z_j - z_{\max}}} = \frac{e^{-z_{\max}} e^{z_i}}{e^{-z_{\max}} \sum_{j=1}^{K} e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}},$$

which is also the one used in the application. In this way all the exponents are non-positive and at least one of them is zero: the exponentials are at most 1, and their sum lies between 1 and K. For example, $z = (1000, 998, 997)$ becomes $(0, -2, -3)$, giving the same probabilities as in the previous example without having to compute e^{1000}.

This form does not, however, prevent very small probabilities p_i from being rounded to zero (underflow). Appendix C explains how this affects the computation of cross-entropy. For the numerical analysis, see [1].

A variant of the softmax, not used in the application, also introduces a temperature that changes how concentrated the probabilities are. Appendix A describes its meaning and its formal connection with the Boltzmann distribution.

3.4 Cross-entropy

Now consider a problem in which each example belongs to exactly one of K classes. In the output layer, the network applies the softmax to the pre-activation values and produces a vector $p = (p_1, \dots, p_K)$: every component is positive and their sum is 1. As noted earlier, these values express the probabilities that the model assigns to the various classes, and together they form a *probability distribution* over the classes.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Let c denote the correct class of the example. We represent it by a target vector $t = (t_1, \dots, t_K)$, called *one-hot*: $t_c = 1$ and all the other components are zero. So t is also a probability distribution, concentrated on the observed class.

To train the network with gradient descent, counting misclassifications is not enough. If, for example, a small change in the parameters increased the probability assigned to the correct class without yet making it the most probable one, the count of misclassifications would not register this improvement. As the parameters vary, in fact, the number of misclassifications is piecewise constant and changes in jumps: its gradient is therefore zero on the constant pieces and undefined at the jumps. What we need is a loss function that takes the assigned probabilities into account and, through its gradient, indicates how to update the parameters.

To measure the loss, we can assign the observed class the penalty $-\ln p_c$: it is small when p_c is close to 1 and grows as p_c decreases. The reasons for choosing the logarithm in particular are developed in §3.5.

This function is a special case of *cross-entropy*. The general definition applies to any target distribution t with non-negative components summing to 1. Each class q is assigned the penalty $-\ln p_q$, and the weighted average of these penalties is taken, using as weights the probabilities t_q of the target distribution:

$$E = -\sum_{q=1}^{K} t_q \ln p_q. \tag{3}$$

So p_q determines the penalty associated with class q, while t_q sets its weight in the average.

In our problem, the one-hot target gives weight 1 to the observed class and weight 0 to all the others. The weighted average thus reduces to a single term:

$$E = -\ln p_c. \tag{4}$$

The general form (3) is also the starting point for deriving the corresponding error signal $\delta = p - t$ in §5.3.3.

The loss of a single example therefore depends only on p_c. However, the probabilities produced by the softmax are tied together by the constraint $\sum_i p_i = 1$: a change in the total probability assigned to the other classes therefore also produces an opposite change in p_c. A redistribution among the other classes that leaves their sum unchanged, on the other hand, leaves p_c and E unchanged.

The loss function $-\ln p_c$ is decreasing. With at least two classes and finite pre-activation values, the softmax gives $0 < p_c < 1$, so the loss is positive, tends to $+\infty$ as p_c tends to 0^+, and tends to 0 as p_c tends to 1. A loss of zero is therefore a limiting value.

p_c	$E = -\ln p_c$
0.99	0.010
0.9	0.105
0.5	0.693
0.1	2.303
0.01	4.605

Assigning a very low probability to the observed class thus leads to a high penalty: this is what happens, in particular, when the model assigns a probability close to 1 to a wrong class. In general, every time p_c shrinks by a factor of 10, the loss increases by $\ln 10 \approx 2.303$.

Note that cross-entropy and the percentage of correct classifications measure different things. For example, if p_c rises from 0.6 to 0.9, the predicted class stays correct, but the loss drops from about 0.511 to 0.105. During training,

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

increasing the probabilities assigned to the correct classes can therefore reduce the loss even when the number of correct classifications does not change.

3.5 The connection with maximum likelihood

Consider N training examples, with inputs x_r and observed classes c_r. Let θ denote the set of weights and biases, and $p_{c_r}(x_r;\theta)$ the probability that the network assigns to the observed label of example r.

Given the inputs and the model parameters, we assume that the labels of the different examples are independent. The probability that the model assigns to the whole sequence of observed labels is then obtained by multiplying the probabilities of the individual examples:

$$\mathcal{L}(\theta) = \prod_{r=1}^{N} p_{c_r}(x_r;\theta). \tag{5}$$

The examples and their labels stay fixed; as the parameters θ vary, the probabilities assigned by the network change, and so does the product $\mathcal{L}(\theta)$. Regarded as a function of the parameters, this quantity is called the *likelihood*, and the maximum likelihood criterion consists in looking for the parameters that maximize it.

Since the logarithm is increasing, maximizing $\mathcal{L}$ is equivalent to maximizing $\ln \mathcal{L}$. Changing the sign and dividing by N, we obtain the objective to be minimized:

$$\overline{E}(\theta) = -\frac{1}{N}\ln \mathcal{L}(\theta) = -\frac{1}{N}\sum_{r=1}^{N} \ln\, p_{c_r}(x_r;\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r(\theta). \tag{6}$$

Minimizing the average cross-entropy is therefore equivalent to maximizing the likelihood of the labels under the model. The application tries to reduce this average loss (§7) by updating the parameters after each example.

The logarithm also has an interpretation in terms of the information associated with an event: Appendix B develops this reading, introduces Shannon entropy, and explains the name “cross-entropy”. These concepts round out the interpretation of the loss function, but they are not needed to derive its gradient.

4. Gradient descent

Once the loss function has been chosen, we still need to work out how to change the parameters. For a dataset of N examples, let $E_r(\theta)$ denote the loss on example r and $\overline{E}(\theta)$ the average:

$$\bar{E}(\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r(\theta),$$

where we again collect the parameters in a vector $\theta = (\theta_1, \dots, \theta_P)$. The gradient $g = \nabla\overline{E}(\theta)$ is the vector of partial derivatives: its j-th component gives the local rate of change of the average loss with respect to the parameter θ_j, with the other parameters held fixed. If the function is differentiable at the point considered, then for a small displacement h the first-order approximation holds:

$$\overline{E}(\theta + h) = \overline{E}(\theta) + g^T h + o(\|h\|).$$

Here $g^T h = \sum_{j=1}^{P} g_j\, h_j$ is the dot product, $\|h\|$ is the length of the displacement vector, and $o(\|h\|)$ denotes a remainder that, divided by $\|h\|$, tends to zero as h tends to the zero vector.

Setting $h = -\eta g$, with the point θ fixed and for $\eta \to 0^+$, we obtain

$$\overline{E}(\theta - \eta g) = \overline{E}(\theta) - \eta\|g\|^2 + o(\eta).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

For $g \neq 0$ and η small enough, the negative term dominates the remainder and the loss decreases. The positive number η is the learning rate: it controls the size of the step. A step that is too large can increase the loss, because the local approximation is no longer valid; a very small one can make training slow. At points where the gradient is zero, this argument gives no direction of decrease and does not guarantee that a minimum has been reached.

For a single example, the same rule is applied to its loss E. A generic parameter θ_j, which can be a weight or a bias, is updated according to

$$\theta_{j,\text{new}} = \theta_j - \eta \frac{\partial E}{\partial \theta_j}, \tag{7}$$

with all the derivatives evaluated at the current values of the parameters. Reducing the loss on one example does not mean that the average loss over the dataset decreases at every step: this distinction is examined in §7.

Gradient descent determines how the derivatives are used to update the parameters. Backpropagation, on the other hand, determines how to compute them efficiently, by applying the chain rule from the output back toward the earlier layers and reusing derivatives that have already been computed. They are two distinct operations within the same training process.

5. Backpropagation

5.1 The δ term

As we saw in equation (7), updating the weights and biases by gradient descent requires the partial derivatives

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} \quad \text{and} \quad \frac{\partial E}{\partial b_i^{(\ell)}}.$$

To organize this computation, we associate with each neuron a quantity that will be reused when the derivatives are computed backward through the network (the *backward pass*):

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

The symbol $\delta_i^{(\ell)}$, often informally called the **error signal**, thus represents the derivative of the loss E with respect to the pre-activation value $z_i^{(\ell)}$ of the corresponding neuron; it expresses the local, signed sensitivity of the loss to changes in the pre-activation.

When the activation is applied component by component, since

$$a_i^{(\ell)} = f_\ell\left(z_i^{(\ell)}\right),$$

the chain rule also gives

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial a_i^{(\ell)}} f_\ell'\left(z_i^{(\ell)}\right).$$

In the next section, we will use $\delta_i^{(\ell)}$ to compute the derivatives of the loss with respect to the neuron's incoming weights and bias.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

5.2 Derivative with respect to a weight and a bias

Consider a specific weight, $w_{ij}^{(\ell)}$. In the sum

$$z_i^{(\ell)} = \sum_{q=1}^{n_{\ell-1}} w_{iq}^{(\ell)}\, a_q^{(\ell-1)} + b_i^{(\ell)},$$

this weight appears only in the term $w_{ij}^{(\ell)} a_j^{(\ell-1)}$. Therefore

$$\frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = a_j^{(\ell-1)}.$$

Applying the chain rule:

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \frac{\partial E}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}. \tag{8}$$

For the bias,

$$\frac{\partial z_i^{(\ell)}}{\partial b_i^{(\ell)}} = 1,$$

and therefore

$$\frac{\partial E}{\partial b_i^{(\ell)}} = \delta_i^{(\ell)}. \tag{9}$$

The two relations can thus be summarized as follows, where “to” and “from” refer to the neurons at the two ends of the connection:

$$\frac{\partial E}{\partial w} = \delta_{\text{to}}\, a_{\text{from}}, \qquad \frac{\partial E}{\partial b} = \delta_{\text{neuron}}.$$

5.3 The δ term in the output layer

The values of $\delta_i^{(L)}$ for the output neurons are obtained analytically from the loss function and the output transformation: in this section we look in detail at the cases implemented in the application. In the hidden layers, by contrast, the values of $\delta_i^{(\ell)}$ are computed recursively from those of the next layer, as described in §5.4.

5.3.1 Sigmoid and squared error

With a single output, using a sigmoid and the squared error as in the application, we have

$$a = \hat{y} = \sigma(z), \qquad E = \frac{1}{2}(a - y)^2.$$

Since

$$\frac{\partial E}{\partial a} = a - y \qquad \text{and} \qquad \sigma'(z) = a(1 - a),$$

we obtain

$$\delta^{(L)} = (a - y)\, a(1 - a). \tag{10a}$$

In practice, however, binary classification commonly pairs the sigmoid with binary cross-entropy; in that case the output term simplifies to $\delta^{(L)} = a - y$. The two loss functions are compared in §6.1.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

5.3.2 Linear output and squared error

If the last layer uses $a = z$, then $f'(z) = 1$ and therefore

$$\delta^{(L)} = a - y\,. \tag{10b}$$

5.3.3 Softmax and cross-entropy

We now combine the softmax from §3.2 with the loss function from §3.4 to derive an expression for the gradient with respect to the output pre-activations.

If $z_i^{(L)}$ is the pre-activation value of output neuron i, the softmax gives

$$p_i = \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}}.$$

We use one-hot encoding, in which t_i denotes the target value: $t_i = 1$ for the correct class and $t_i = 0$ for all the others. The cross-entropy of a single example is

$$E = -\sum_{q=1}^{n_L} t_q \ln p_q = -\ln p_c,$$

where c is the correct class. In the softmax, each probability depends on all the output pre-activations. Applying the chain rule through the probabilities would require keeping track of this dependence. The computation becomes simpler if we first express the loss directly in terms of the pre-activations. Since

$$\ln p_q = z_q^{(L)} - \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right)$$

and $\sum_i t_i = 1$, we can rewrite

$$E = -\sum_{q=1}^{n_L} t_q z_q^{(L)} + \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right).$$

Differentiating with respect to $z_i^{(L)}$ then gives

$$\delta_i^{(L)} = \frac{\partial E}{\partial z_i^{(L)}} = -t_i + \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}} = p_i - t_i. \tag{10c}$$

For the correct class we therefore have $\delta_c^{(L)} = p_c - 1 < 0$; for every other class we have $\delta_i^{(L)} = p_i > 0$. Treating the values $z_i^{(L)}$ as variables, the descent direction therefore raises the value for the correct class and decreases those for the other classes.

For the parameters of the output layer, the following relations hold:

$$\frac{\partial E}{\partial w_{ij}^{(L)}} = \delta_i^{(L)} a_j^{(L-1)}, \qquad \frac{\partial E}{\partial b_i^{(L)}} = \delta_i^{(L)}.$$

The updates are obtained by multiplying these derivatives by $-\eta$. Even though each component of $\delta^{(L)}$ lies in $(-1,1)$, the size of the update therefore depends on the learning rate and, for the weights, on the activations of the previous layer.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

5.4 The δ term in a hidden layer

Consider neuron i in a hidden layer ℓ. Its activation $a_i^{(\ell)}$ reaches all the neurons $k = 1, \dots, n_{\ell+1}$ of the next layer. For each of them,

$$z_k^{(\ell+1)} = \sum_{q=1}^{n_\ell} w_{kq}^{(\ell+1)} a_q^{(\ell)} + b_k^{(\ell+1)}.$$

We want to compute

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

The loss depends on $z_i^{(\ell)}$ through the activation $a_i^{(\ell)}$ and then through all the subsequent neurons. The chain rule must therefore sum over all these paths:

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} \frac{\partial E}{\partial z_k^{(\ell+1)}} \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}}.$$

But

$$\frac{\partial E}{\partial z_k^{(\ell+1)}} = \delta_k^{(\ell+1)} \qquad \text{and} \qquad \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}} = w_{ki}^{(\ell+1)},$$

so

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \, \delta_k^{(\ell+1)}.$$

Finally, $a_i^{(\ell)} = f_\ell\left(z_i^{(\ell)}\right)$, so

$$\delta_i^{(\ell)} = f_\ell{}'\left(z_i^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}. \tag{11}$$

This recursive formula therefore makes it possible to compute the term δ of a hidden layer from the terms δ of the next layer (Figure 5).

As a mnemonic, the structure of the recursion is therefore:

$$\delta_{\text{neuron}} = f'(z_{\text{neuron}}) \cdot \sum_{\text{next neurons}} (w \cdot \delta_{\text{next}}).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

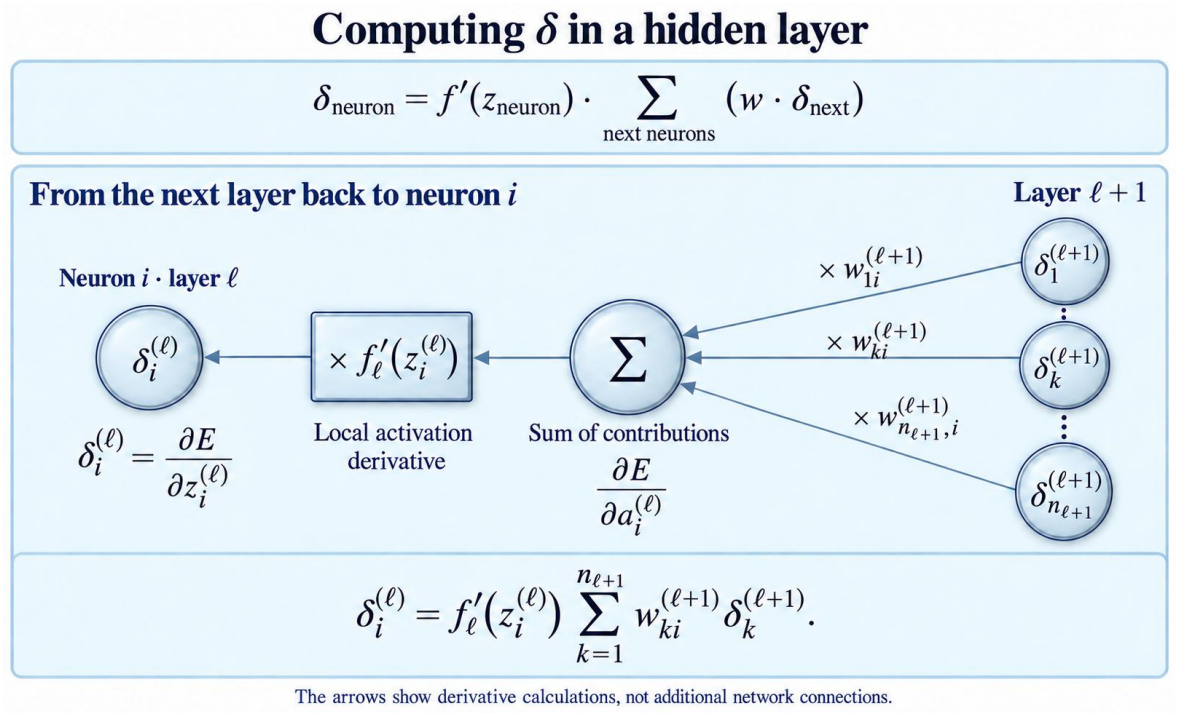


Figure 5. Backward computation of $\delta_i^{(\ell)}$ in a hidden layer. The contributions $w_{ki}^{(\ell+1)}\delta_k^{(\ell+1)}$ of the next-layer neurons add up to $\partial E/\partial a_i^{(\ell)}$; multiplying by $f_\ell'(z_i^{(\ell)})$ gives $\delta_i^{(\ell)}$, as in (11). This is the neuron diagram of Figure 3, traversed backward.

5.5 The procedure for a single example

For a single example, with the inputs and the target fixed:

1. **Forward pass.** Compute and store all the $z_i^{(\ell)}$ and $a_i^{(\ell)}$ using (1) and (2), with the softmax in the multiclass output layer (§3.2).
2. **Loss.** Compute E from the network's output and the target.
3. **Output deltas.** Compute the $\delta_i^{(L)}$ according to the loss function and the output transformation. In the cases handled by the application, use (10a), (10b), or (10c).
4. **Hidden deltas.** Compute the $\delta_i^{(\ell)}$ from the last layers back to the first by applying (11) recursively.
5. **Derivatives with respect to the parameters.** Once all the terms $\delta_i^{(\ell)}$ are known, the derivatives of E with respect to the weights and biases are given by (8) and (9).
6. **Update.** Compute the parameter changes using (7) and apply them all together.

All the derivatives must refer to the same state of the network: this is why the derivatives of E with respect to all the weights and biases are computed first, and only then are the updates applied together.

6. How the loss function affects the gradient

6.1 Comparison with the squared error

The binary case shows the role of the loss function with a single output $a = \sigma(z)$ and a target $y \in \{0,1\}$. The probability assigned to class 1 is a, and the probability assigned to class 0 is $1 - a$. The *binary cross-entropy* (BCE) then becomes

$$E_{\text{BCE}} = -y \ln a - (1 - y) \ln (1 - a).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

The comparison is as follows:

Loss function	Derivative with respect to z
$E_{\text{quad}} = \frac{1}{2}(a-y)^2$	$\delta = (a-y)a(1-a)$
$E_{\text{BCE}} = -y\ln a - (1-y)\ln(1-a)$	$\delta = a - y$

The BCE is obtained from the general definition of cross-entropy (3) by setting $p = (a, 1-a)$ and $t = (y, 1-y)$, with the components ordered as class 1 and class 0. The relationship between the output transformations is just as direct:

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z + e^0}.$$

The sigmoid therefore returns the first component of the softmax applied to $(z, 0)$; the second component is $1 - a$. Treating z as the only variable, the first component of relation (10c) gives back $\delta = a - y$.

With the squared error, the factor $a(1-a)$ appears, which is the derivative of the sigmoid. This factor tends to zero as a approaches 0 or 1, even when the prediction is wrong. The sigmoid is said to be *saturated* when it is in one of these almost flat regions of its graph: a change in z then produces only a very small change in the output a.

For $y = 0$ we have $|\delta_{\text{quad}}| = a^2(1-a)$; for $y = 1$, $|\delta_{\text{quad}}| = a(1-a)^2$. Finding the maximum of these two cubics on the interval $[0,1]$ gives the bound $|\delta_{\text{quad}}| \leq 4/27 \approx 0.148$.

For example, with $a = 0.99$ and $y = 0$ we obtain

$$\delta_{\text{quad}} = 0.99 \cdot 0.99 \cdot 0.01 = 0.009801, \qquad \delta_{\text{BCE}} = 0.99.$$

In this situation the derivative with cross-entropy is about a hundred times larger. As a tends to 0 with $y = 1$, the derivative obtained with the squared error tends to 0, whereas the one obtained with binary cross-entropy tends to -1. Cross-entropy therefore prevents the saturation of the output sigmoid from weakening the signal precisely when the prediction is badly wrong. The same happens in the multiclass case: as p_c tends to zero, $\delta_c^{(L)}$ tends to -1. This does not, however, prevent the gradient from being weakened in the hidden layers.

The two functions also differ in the range of their values. In the binary case just defined, $E_{\text{quad}} \leq \frac{1}{2}$, whereas if the squared error is applied to a whole multiclass vector, in the form $\frac{1}{2}\sum_i (p_i - t_i)^2$ with a one-hot target, the upper bound is 1. Cross-entropy, on the other hand, has no upper bound.

The different upper bound of the squared error depends on the chosen representation. With two classes, $p = (a, 1-a)$ and $t = (y, 1-y)$, half the sum of the two squared deviations is $(a-y)^2$: twice the scalar loss $\frac{1}{2}(a-y)^2$. The bounds 1 and $1/2$ are therefore consistent; they do not reflect any real difference between the two-class and the multiclass case.

In the application, cross-entropy is used for multiclass classification, while the squared error is kept for binary classification as a teaching simplification. Both options are mentioned in §5.3.1.

6.2 An example with three classes

Let us return to the example used in §3.2 and in the application's Activation functions guide:

$$z^{(L)} = (2,\ 0,\ -1).$$

The softmax gives

$$p \approx (0.843795,\ 0.114195,\ 0.042010).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

The loss depends on the observed class:

- **if the correct class is the first one**, the target is $t = (1,0,0)$:
$$E = -\ln p_1 \approx 0.170, \qquad \delta^{(L)} \approx (-0.156,\ 0.114,\ 0.042).$$
- **if the correct class is the third one**, the target is $t = (0,0,1)$:
$$E = -\ln p_3 \approx 3.170, \qquad \delta^{(L)} \approx (0.844,\ 0.114,\ -0.958).$$

In the second case, the network assigned only about 4.2% to the correct class. Since the target is $t_3 = 1$, we have $\delta_3 = p_3 - 1 \approx -0.958$, close to the limiting value −1 that δ_3 approaches as p_3 tends to zero: as discussed in §5.3.3, the derivative does not shrink precisely when the prediction is badly wrong.

7. From a single example to the dataset

7.1 The average gradient and sampling

The goal of training is to minimize the average loss $\overline{E}(\theta)$ introduced in §4. The derivatives computed for individual examples can be combined to obtain the derivatives of this average.

Since differentiation is linear, the gradient of the average loss with respect to the parameters is the average of the gradients $\nabla E_r(\theta)$ of the losses on the individual examples:

$$\nabla \bar{E}(\theta) = \frac{1}{N}\sum_{r=1}^{N} \nabla E_r(\theta).$$

Gradient descent on the whole dataset. The gradients of the losses on all the examples in the dataset are computed in the same state of the network, they are averaged, and a single update is applied:

$$\theta_{\text{new}} = \theta - \eta\, \nabla \bar{E}(\theta).$$

This procedure is called *batch gradient descent*.

Stochastic gradient descent (SGD). It is called stochastic because at each step the update direction depends on an example chosen at random, instead of being computed from the whole dataset at once. In the case of independent sampling, let R denote the index of the example drawn at a given step: at each step, R is chosen uniformly and independently from $1, \dots, N$, and the parameters are updated immediately:

$$\theta_{\text{new}} = \theta - \eta\, \nabla E_R(\theta).$$

Since each index has probability $1/N$, for fixed parameters the expected value of the gradient $\nabla E_R(\theta)$ is:

$$\mathbb{E}_R[\nabla E_R(\theta)] = \sum_{r=1}^{N} \frac{1}{N}\, \nabla E_r(\theta) = \frac{1}{N}\sum_{r=1}^{N} \nabla\, E_r(\theta) = \nabla \bar{E}(\theta).$$

Here the expected value is taken with respect to the random choice of R among the N examples of the given dataset. In this sense, the gradient of the loss on the sampled example is an unbiased estimator of the gradient of the average loss. As a result, for fixed parameters, the average update of an SGD step is the same as that of a *batch gradient descent* step with the same rate η, even though an individual update may differ.

7.2 Epochs and mini-batches

An epoch is one pass through all the training examples.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Random reshuffling. At the start of each epoch, a new random order $r_1, \dots, r_N$ of the examples is generated, and the examples are then used once each: sampling is therefore done without replacement. This is the algorithm used by the application.

Since the parameters change after each update, each example is processed in the state left by the previous ones. The gradients in an epoch are therefore not all computed at the same point, and their average generally differs from the average gradient computed at the start: the independence argument of §7.1 cannot simply be carried over to sampling without replacement.

For a small learning rate, the difference from the full batch can be quantified: Appendix D develops the first-order comparison under suitable regularity assumptions.

Mini-batches. In practice, an intermediate solution between batch gradient descent and single-example updates is often used. For a group B of m examples we compute

$$g_B(\theta) = \frac{1}{m} \sum_{r \in B} \nabla E_r(\theta)$$

and update the network using $g_B(\theta)$. If B is chosen uniformly for fixed parameters, $g_B(\theta)$ is an unbiased estimator of the gradient of the average loss. Because it averages the contributions of several examples, this quantity usually varies less from one sample to the next than the gradient of the loss on a single example does: it is therefore a less noisy estimate of the gradient of the average loss over the whole dataset. Compared with the full batch, it requires less computation per update and makes efficient use of parallel computing. In practice, mini-batches are often formed by shuffling the dataset at the start of each epoch, splitting it into groups, and updating the parameters after each group. Updating on a single example corresponds to the case $m = 1$; the full batch corresponds to the case $m = N$. The application uses $m = 1$ together with random reshuffling, without replacement within each epoch.

8. From training to generalization

8.1 The scale of the inputs

The formulas above assume that the inputs are already expressed as numbers. The scale of these numbers, however, affects the computations: in the first layer, for example, the derivative with respect to a weight contains the factor x_j (equation (8)). Inputs of very different orders of magnitude can therefore produce unbalanced gradients and make it harder to choose a single learning rate.

The application brings the inputs to a common scale with a min–max transformation. If X varies in an interval $[X_{\min}, X_{\max}]$ of positive length, we set

$$x = \frac{X - X_{\min}}{X_{\max} - X_{\min}}.$$

This affine transformation maps the endpoints to 0 and 1. The scale can be fixed by the domain of the problem, as in the simulator's built-in problems, or derived from the training data alone. In the latter case, the same endpoints must then be used for the validation data and for new predictions: recomputing them would change the meaning of the inputs. A new data point outside the original interval can give a normalized value outside $[0,1]$.

8.2 Why the weights are initialized at random

Before training, the parameters must be given initial values. If two hidden neurons with the same activation started out in perfectly symmetric conditions — the same incoming weights, the same bias, and the same weights toward each neuron in the next layer — they would produce the same activations and receive the same updates. They would thus remain duplicates throughout training.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

If all the weights and biases were initially zero, then in the first backpropagation pass the weighted sums in formula (11) would vanish, and so would the δ terms of the hidden layers. This does not mean that all the weights would stay at zero: with the sigmoid, for example, the hidden activations are initially $1/2$, and the weights of the output layer can already change at the first update. The symmetry, however, remains: the neurons in the same hidden layer would keep receiving identical updates and would never become different from one another.

Random initialization of the weights breaks this symmetry and lets the neurons evolve differently. In the application, the weights are initialized uniformly between -1 and 1 and the biases to zero: the biases do not need to be random as well for the neurons to differentiate. In larger networks the scale of the initial weights also matters, and it is chosen according to the size of the layers and the activation function. The simulator's fixed seed makes it possible to reproduce the same initialization and compare experiments.

8.3 Training loss and loss on new data

Gradient descent uses the available examples to reduce the training loss. What we actually want, however, is a model that also makes useful predictions on new examples of the same problem: this ability is called generalization. A small training loss does not, by itself, guarantee it.

To assess it, a set of examples is held out for validation and not used in the weight updates. If the training loss keeps decreasing while the validation loss increases, the model may be fitting the peculiarities of the training sample as well: this is called overfitting. The simulator's curves let you observe this distinction; a single fluctuation, however, is not enough to diagnose the phenomenon.

If the validation data repeatedly guide the choice of architecture or of when to stop training, they indirectly become part of the model selection process. An independent final evaluation would then require additional test data. Moreover, the observed performance remains tied to the kind of data on which it is measured: new data very different from the training data may call for a different model.

9. From index notation to matrix form

The equations written in index notation can be expressed in vector and matrix form by writing them for all the neurons in a layer at once.

For a layer $\ell = 1, \dots, L$ we define:

- $z^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of pre-activation values.
- $a^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of output activations.
- $a^{(0)} = x$ is the column vector of inputs, as in §1.3.
- $b^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of biases.
- $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ is the weight matrix. Row i contains the weights of the connections arriving at neuron i of layer ℓ; column j contains the weights of the connections leaving node j of layer $\ell - 1$. In particular, $\left(W^{(\ell)}\right)_{ij} = w_{ij}^{(\ell)}$.

9.1 Forward pass

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)},$$
$$a^{(\ell)} = f_\ell\left(z^{(\ell)}\right).$$

In the application, the function f_ℓ is applied component by component, except for the softmax in the output layer.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

9.2 Backward pass in the hidden layers

For a hidden layer ($1 \le \ell \le L-1$), we collect the terms $\delta_i^{(\ell)}$ in a column vector:

$$\delta^{(\ell)} = \begin{pmatrix} \delta_1^{(\ell)} \\ \vdots \\ \delta_{n_\ell}^{(\ell)} \end{pmatrix} \in \mathbb{R}^{n_\ell}.$$

For the i-th component, the formula derived in §5.4 is

$$\delta_i^{(\ell)} = f_\ell'\left(z_i^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

The sum on the right is the i-th component of the matrix product $\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}$:

$$\left[\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right]_{i,1} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

Writing all the components at once, we therefore obtain

$$\delta^{(\ell)} = \left(\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right) \odot f_\ell'\left(z^{(\ell)}\right),$$

where $\odot$ denotes the component-wise product, also called the Hadamard product.

9.3 Derivatives with respect to the parameters

Let $\frac{\partial E}{\partial W^{(\ell)}}$ denote the $n_\ell \times n_{\ell-1}$ matrix defined by

$$\left[\frac{\partial E}{\partial W^{(\ell)}}\right]_{ij} = \frac{\partial E}{\partial w_{ij}^{(\ell)}}.$$

By equation (8), we have

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

The right-hand side of equation (8) is the entry in position (i, j) of the product of the column vector $\delta^{(\ell)}$ and the row vector $\left(a^{(\ell-1)}\right)^T$. We can therefore collect all the derivatives of E with respect to the weights in the matrix

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)}\left(a^{(\ell-1)}\right)^T.$$

For the biases, defining $\frac{\partial E}{\partial b^{(\ell)}}$ as the column vector with components $\partial E / \partial b_i^{(\ell)}$, equation (9) gives the vector equality

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

9.4 Extension to mini-batches

So far, $z^{(\ell)}$, $a^{(\ell)}$, and $\delta^{(\ell)}$ have been vectors for a single example. For a mini-batch of m examples, we place the vectors $a^{(\ell-1)}$ side by side as the columns of $A^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1} \times m}$, and the vectors $\delta^{(\ell)}$ as the columns of $\Delta^{(\ell)} \in \mathbb{R}^{n_\ell \times m}$. The columns correspond to the same examples, in the same order; the δ terms are the derivatives of the individual losses. All the values are computed with the same parameters, before the update. The average of the gradients with respect to $W^{(\ell)}$ is then

$$\frac{1}{m} \Delta^{(\ell)} (A^{(\ell-1)})^T.$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

The entry in position (i, j) of this matrix is $\frac{1}{m}\sum_{r=1}^{m} \Delta_{ir}^{(\ell)} A_{jr}^{(\ell-1)}$: the product adds up the contributions of the individual examples, and the factor $1/m$ turns the sum into their average. The forward pass, the backward pass, and the computation of the gradients thus become largely matrix operations, which are particularly well suited to parallel execution on GPUs.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Appendix A. Temperature and the Boltzmann distribution

A parameter $\tau > 0$, called the temperature, can be introduced by dividing the pre-activations by τ before applying the softmax [2]:

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_{j=1}^{K} e^{z_j/\tau}} = \frac{e^{(z_i - z_{\max})/\tau}}{\sum_{j=1}^{K} e^{(z_j - z_{\max})/\tau}}.$$

For fixed logits, the ratio $\frac{p_i(\tau)}{p_j(\tau)} = e^{(z_i - z_j)/\tau}$ shows the role of the parameter: lowering the temperature accentuates the differences, while raising it pushes the ratios toward 1 and makes the distribution more uniform. For every positive temperature, the order of the probabilities, and hence the most probable class, stays the same.

As $\tau \to 0^+$, if the maximum is unique, the probability of the corresponding class tends to 1 and the others to 0. If the maximum is attained by s classes, each of them tends instead to $1/s$. As $\tau \to \infty$, all the probabilities tend to $1/K$. These limits are understood for a finite number of classes and fixed logits.

For example, for $z = (2, 0, -1)$:

τ	$p_1(\tau)$	$p_2(\tau)$	$p_3(\tau)$
0.5	0.979629	0.017943	0.002428
1	0.843795	0.114195	0.042010
2	0.628532	0.231224	0.140244

As the temperature increases, the network gives the favored class less of an edge over the alternatives. If the output with the highest probability is always chosen, changing the temperature does not change the choice. In text generation with language models (LLMs), however, the next token — for example a word or part of a word — can be sampled at random according to the computed probabilities, possibly restricting the sampling to the most probable candidates. Raising the temperature makes the distribution less concentrated and, overall, gives more room to the alternatives to the favored token: it can therefore produce less predictable continuations, even though, for a given context, the order of the probabilities remains unchanged.

In statistical mechanics, a system in equilibrium with an environment at absolute temperature $T > 0$ occupies a state i with energy ε_i with probability given by the Boltzmann distribution [3]:

$$P_i(T) = \frac{e^{-\varepsilon_i/(k_B T)}}{Z(T)}, \qquad Z(T) = \sum_j e^{-\varepsilon_j/(k_B T)}.$$

Here k_B is the Boltzmann constant and $Z(T)$, called the partition function, is the sum that normalizes the probabilities. Each index identifies a state; several states can have the same energy.

The form is the same as that of the softmax with temperature: after choosing an energy scale $\varepsilon_0 > 0$, it suffices to set $z_i = -\varepsilon_i/\varepsilon_0$ and $\tau = k_B T/\varepsilon_0$. Larger pre-activations thus correspond to lower energies. A low temperature favors the states of lowest energy; a higher temperature reduces the weight of energy differences.

In the network, however, the logits are values computed by the model and τ is a mathematical parameter, with no necessary thermodynamic meaning.

The relation $\delta = p - t$ refers to $\tau = 1$. With the softmax at temperature τ introduced in this appendix and a fixed target, the same derivation gives $\frac{\partial E}{\partial z_i} = \frac{p_i(\tau) - t_i}{\tau}$. Both the probabilities and the factor appearing in the gradient therefore change.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Appendix B. Entropy and information

B.1 Information and surprise

An event to which we assign a high probability is expected; one to which we assign a low probability is unexpected. Information theory expresses this idea by assigning to the observation of an event with probability p the value $-\ln p$, called the *information content* of the event, or its *surprise*. Here "surprise" therefore denotes a mathematical quantity, computed from the probability assigned to the event. It is 0 for a certain event and increases as the event's probability decreases.

Choosing the logarithm also makes the information of independent events additive: if two events have probabilities p and p', the probability that both occur is pp', so the surprise associated with their joint occurrence is

$$-\ln(pp') = -\ln p - \ln p'.$$

The penalty $-\ln p_c$ introduced in §3.4 therefore coincides with the surprise associated with the observed label according to the model's probabilities. Reducing the average cross-entropy amounts to reducing the average surprise associated with the labels of the training examples. The reason for the name "cross-entropy" is discussed in the next section, §B.2.

B.2 Shannon entropy and Kullback–Leibler divergence

To give meaning to the name *cross-entropy*, consider an experiment with K possible outcomes. In this section $t = (t_1, \dots, t_K)$ denotes a generic distribution: outcome q occurs with probability t_q.

If we know these probabilities, we associate with the observation of outcome q the information, or surprise, $-\ln t_q$, in the sense defined in §B.1. To compute the average surprise, we weight each value $-\ln t_q$ by the probability t_q that the outcome occurs. This weighted average is the *Shannon entropy*:

$$H(t) = -\sum_{q=1}^{K} t_q \ln t_q.$$

Entropy therefore represents the average information provided by observing the outcome. The same quantity measures the uncertainty before the observation: if the outcome is already certain, observing it provides no new information and the entropy is zero.

An outcome with probability zero does not contribute to the average: the formula uses the convention $0 \ln 0 = 0$, consistent with the limit $u \ln u \to 0$ as $u \to 0^+$.

For example, a fair coin has two outcomes, heads and tails, each with probability $1/2$. The surprise associated with each outcome is $-\ln(1/2) = \ln 2$, so the average surprise is also $H(t) = \ln 2 \approx 0.693$.

Now suppose the outcomes still occur with probabilities t_q, but a model assigns them probabilities p_q. When outcome q is observed, the surprise computed by the model is $-\ln p_q$. In averaging it to obtain $H(t, p)$, however, we still use the weights t_q, because these describe how often we expect the various outcomes to occur; hence the name *cross-entropy*:

$$H(t, p) = -\sum_{q=1}^{K} t_q \ln p_q.$$

In this sum, too, outcomes with $t_q = 0$ do not contribute and the corresponding terms are omitted. If the model assigns probability zero to an outcome that can occur, the cross-entropy is infinite; but, as we saw in §3.4, with the softmax and finite pre-activations this cannot happen, except as a result of numerical rounding (Appendix C).

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

In the fair coin example, imagine a model that assigns probability 0.9 to heads and 0.1 to tails. The model finds heads unsurprising and tails very surprising; the coin, however, produces the two outcomes with equal probability. The model's average surprise is therefore

$$H(t,p) = -\frac{1}{2}\ln 0.9 - \frac{1}{2}\ln 0.1 \approx 1.204.$$

Compared with the model that uses the correct probabilities, the average surprise in this example has increased by about $1.204 - 0.693 = 0.511$.

In general, the difference

$$D_{\mathrm{KL}}(t||p) = \sum_{q:t_q>0} t_q \ln \frac{t_q}{p_q} = H(t,p) - H(t)$$

is called the *Kullback–Leibler divergence*. It measures how much the average surprise increases when we use the probabilities p in place of the experiment's probabilities t. This difference is always non-negative and is zero only when $p = t$.

In the case of the network, the target t of a single example is one-hot, so its entropy is zero: the only term to consider is $-1\ln 1 = 0$. Consequently,

$$E = H(t,p) = D_{\mathrm{KL}}(t||p) = -\ln p_c.$$

Reducing the loss on a single example is therefore equivalent to reducing the divergence between the target distribution and the distribution predicted by the network.

With the natural logarithm, the entropy $H(t)$, the cross-entropy $H(t,p)$, and the Kullback–Leibler divergence $D_{\mathrm{KL}}(t||p)$ are measured in *nats*. With base 2 they are measured in *bits*, and all the values are divided by $\ln 2$. The minimum points and the directions of the gradients stay the same; what changes is their scale and, for the same learning rate, the size of the updates.

Appendix C. Computing cross-entropy numerically

As noted in §3.3, in floating-point arithmetic a very small probability can be rounded to zero (underflow), even when the softmax is computed with the maximum subtracted. The application avoids taking the logarithm of zero by using a lower threshold μ for its argument:

$$E_{\text{computed}} = -\ln\left(\max\left(p_c, \mu\right)\right).$$

The threshold is the JavaScript constant Number.MIN_VALUE, the smallest representable positive number: in common browsers, $\mu = 2^{-1074}$ [4]. Every representable positive value is already at least μ, so the substitution affects only a numerical zero. The computed loss never exceeds $-\ln\mu = 1074\ln 2 \approx 744.440$. This safeguard keeps the result finite, but it does not recover the probability lost in rounding.

The threshold affects only the value of the loss. Backpropagation uses $\delta = p - t$, derived in §5.3.3 for the theoretical cross-entropy and evaluated with the numerically computed probabilities. When the threshold comes into play, the gradient used is therefore not the derivative of the truncated function.

The detour through the probability can be avoided by computing the loss directly from the pre-activations: with the notation of §3.3, substituting the softmax into $E = -\ln p_c$ and applying the properties of the logarithm gives

$$E = -\ln p_c = (z_{\max} - z_c) + \ln\left(\sum_{i=1}^{K} e^{z_i - z_{\max}}\right).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

The sum inside the logarithm lies between 1 and K: each exponential is at most 1 and at least one equals 1. This form avoids the logarithm of zero caused by underflow of p_c.

For clarity of exposition, the application keeps the computation from p_c with the threshold described above. The alternative computation from the pre-activations is present in the code as the commented-out function crossEntropyFromLogits.

Appendix D. One epoch of random reshuffling to first order

Let us return to the comparison of §7.2. Let $r_1, \dots, r_N$ be the random order of the examples during an epoch, and let θ_0 be the parameter vector before the first update. We use $O(\eta)$ to denote a quantity whose norm is bounded by a constant times η as η tends to zero, and similarly for $O(\eta^2)$. The number N of examples is fixed.

If the parameters stayed fixed at their initial value θ_0 during the epoch, the average of the gradients would be exactly the gradient of the average loss, whatever the order of the examples:

$$\frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) = \frac{1}{N}\sum_{r=1}^{N} \nabla E_r(\theta_0) = \nabla \bar{E}(\theta_0).$$

Since every permutation contains each example exactly once, under this assumption the average gradient over the epoch coincides exactly with the average gradient over the dataset.

In random reshuffling with an update after each example, as in the application, the parameters instead change during the epoch. If θ_k denotes the value of the parameters after the k-th update, then

$$\theta_k = \theta_{k-1} - \eta\, \nabla E_{r_k}(\theta_{k-1}).$$

We define the average gradient actually used during the epoch:

$$g_{\text{epoch}} = \frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_{k-1}).$$

To compare it with $\nabla \bar{E}(\theta_0)$, we first observe that, after $k-1$ updates,

$$\theta_{k-1} - \theta_0 = -\eta \sum_{s=1}^{k-1} \nabla E_{r_s}(\theta_{s-1}).$$

For a fixed number of examples, if the gradients remain bounded in the region traversed by the parameters, the sum contains a finite number of terms, and therefore

$$\theta_{k-1} - \theta_0 = O(\eta).$$

If, in addition, the gradient of each E_r varies smoothly enough with the parameters — for example, if it is locally Lipschitz — a change of $O(\eta)$ in the parameters also produces a change of $O(\eta)$ in the gradient:

$$\nabla E_{r_k}(\theta_{k-1}) = \nabla E_{r_k}(\theta_0) + O(\eta).$$

Averaging over the N updates of the epoch, again with N fixed, we obtain

$$g_{\text{epoch}} = \frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) + O(\eta) = \nabla \bar{E}(\theta_0) + O(\eta).$$

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

So the average gradient actually used during an epoch differs from the gradient of the average loss computed at the start of the epoch by $O(\eta)$.

The total update over the epoch is

$$\theta_N = \theta_0 - N\eta\, g_{\text{epoch}} = \theta_0 - N\eta\, \nabla\bar{E}(\theta_0) + O(\eta^2).$$

The main displacement of the parameters is of order η, while the difference from the corresponding batch step is of order η^2. Therefore, for η small enough, a full epoch of random reshuffling with an update after each example produces, to first order, the same update as one step of batch gradient descent on the average loss with learning rate $N\eta$. The factor N appears because an epoch consists of N updates, each using learning rate η, whereas batch gradient descent uses the average gradient. The reshuffling and the intermediate updates determine the higher-order corrections, and hence the exact trajectory.

With the ReLU, the regularity assumption is satisfied, for example, in a neighborhood of the initial parameters in which all the pre-activations of the ReLU neurons, for the examples considered, remain nonzero. The comparison does not automatically apply to steps that cross a point of non-differentiability.

Appendix E. Correspondence with the application's code

The structure of the arrays used in the core of the application corresponds directly to the mathematical notation. In these notes, however, neuron indices start at 1, while JavaScript arrays start at 0; this is why the neuron indices appear as i−1 and j−1 in the table.

Mathematics	Array in the application
$w_{ij}^{(\ell)}$	weights[ℓ−1][i−1][j−1]
$b_i^{(\ell)}$	bias[ℓ−1][i−1]
$z_i^{(\ell)}$	zs[ℓ][i−1]
$a_i^{(\ell)}$	as[ℓ][i−1]
$\delta_i^{(\ell)}$	delta[ℓ][i−1]

In particular, `as[0]` contains the inputs; `zs[ℓ]` and `delta[ℓ]` correspond to layers $\ell = 1, \dots, L$. The arrays `weights[ℓ−1]` and `bias[ℓ−1]`, on the other hand, represent the transition from layer $\ell - 1$ to layer ℓ.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Further reading

[1] P. Blanchard, D. J. Higham, and N. J. Higham, Accurately computing the log-sum-exp and softmax functions, IMA Journal of Numerical Analysis 41(4), 2311–2330, 2021; preprint: arXiv:1909.03469. Analysis of the formulas with the maximum subtracted.

[2] G. Hinton, O. Vinyals, and J. Dean, Distilling the Knowledge in a Neural Network, §§2–2.1. Softmax with temperature and its gradient.

[3] D. Tong, Statistical Physics, §§1.3–1.3.1. The Boltzmann distribution and the partition function.

[4] MDN Web Docs, Number.MIN_VALUE. The smallest positive value in common JavaScript engines.

[5] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, chapters 3, 4, 5, 6, and 8. A general reference for information theory, numerical computation, generalization, feed-forward networks, and training.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.